Name: Anitha Reddy Bapathu

Software Developer

Phone: (518)-350-9016

Email: anithareddybapathu14@gmail.com

LinkedIn **GitHub**



Summary

Software developer with 7+ years of experience building web applications for banking, biopharma, and IT services clients. Day to day that means Java 8 and 17 with Spring Boot, Spring MVC, Spring Security, and Hibernate on the backend and React with Redux and TypeScript on the front end, talking to each other over REST and, where a system needed it, SOAP. Has also built services with Node.js and Express behind React and Angular frontends. Comfortable across the whole delivery cycle in Agile/Scrum teams: understanding requirements, designing, coding, testing, releasing, and supporting production. Has built event-driven services with Kafka, RabbitMQ, and AWS SQS and SNS, secured APIs with JWT, OAuth 2.0, and OpenID Connect, and worked with Oracle, PostgreSQL, MySQL, and MongoDB, including query tuning and stored procedures. Tests with JUnit, Mockito, Jest, Jasmine, Cypress, and Postman, and ships with Maven, Git, Jenkins, SonarQube, Docker, Kubernetes, and AWS. Works closely with QA, DevOps, and business analysts on debugging, performance problems, code reviews, and keeping applications healthy after release.

Technical Skills

Programming Languages	Java 8, Java 17, JavaScript (ES6+), TypeScript, SQL, PL/SQL
Java/J2EE Technologies	Core Java, OOP, Streams, Lambda Expressions, Optional, JPA, JAX-WS, JSON, XML, Exception Handling, Logging
Backend Frameworks	Spring Boot 2.x/3.x, Spring MVC, Spring Core (IoC, DI, AOP), Spring Security, Spring Data JPA, Node.js 16 to 22, Express.js, KafkaJS, Zod
ORM Frameworks	Hibernate, JPA, Spring Data JPA, Sequelize, Mongoose
Frontend Technologies	React 16/18, React Hooks, Redux, Redux Toolkit, React Router, Angular 13–16, RxJS, NgRx, Angular Material, TypeScript, JavaScript (ES6+), HTML5, CSS3, SASS, Bootstrap, Material UI, Axios
Web Services and APIs	RESTful APIs, SOAP, WSDL, JAX-WS, JSON, XML, Swagger/OpenAPI, Postman, SoapUI
Security	Spring Security, JWT, OAuth 2.0, OpenID Connect (OIDC), Role-Based Access Control, Input Validation
Messaging and Streaming	Apache Kafka, RabbitMQ, AWS SQS, AWS SNS
Databases	Oracle, PostgreSQL, MySQL, MongoDB, SQL, PL/SQL, Stored Procedures, Indexing, Query Optimization
Architecture and Design Patterns	MVC, Layered Architecture, Microservices, Event-Driven Architecture, RESTful Service Design, Circuit Breaker, DAO, DTO, Dependency Injection
Cloud Technologies	AWS (EC2, S3, RDS, Lambda, EKS, SQS, SNS, CloudWatch, IAM)
DevOps and CI/CD	Jenkins, CI/CD Pipelines, Docker, Kubernetes, SonarQube

Build and Version Control Tools	Maven, npm, Git, GitHub, Bitbucket
Testing Frameworks	JUnit 5, Mockito, Spring Boot Test, Jest, Supertest, Jasmine, Karma, React Testing Library, Cypress, Postman, SoapUI, Unit Testing, Integration Testing
Application/Web Servers	Apache Tomcat, Nginx
IDEs and Tools	IntelliJ IDEA, Eclipse, Spring Tool Suite, VS Code, Jira, Splunk, Grafana, ESLint, Log4j2, Logback
Methodologies	Agile, Scrum, SDLC, Code Reviews

Professional Experience

Senior Software Engineer **Sep 2025 – Present**
Deutsche Bank

Project description: Payments Exceptions and Reconciliation Workbench: an internal application that payment operations and audit teams use to find failed or unmatched payments, work them through a repair queue, and reconcile the bank's ledger against confirmations from counterparties. Users filter and prioritize exceptions, review the full history of a transaction, and submit repairs, and every manual repair must be approved by a second person before it takes effect. Access is limited by role, and every action is recorded for audit. The application is split into small independent services so teams can release changes to one area without affecting the rest.

Frontend:

- Build the React 18 and TypeScript screens operations staff work in all day: filterable exception queues, a transaction timeline, and approval dialogs, with server-side pagination because the queues can get long.
- Keep shared state such as active filters, the selected item, and user permissions in Redux Toolkit, and keep form state local to each component with Hooks.
- Put together reusable Material UI pieces (data grids, filter panels, status badges, confirm dialogs) so new screens don't start from scratch, and validate repair forms in the browser before they reach the API.
- Call the REST APIs through Axios, with interceptors that attach the access token, refresh an expired session, and turn API errors into readable messages.
- Show or hide actions in React Router and components using the role claims from the JWT, so operators, supervisors, and auditors only see what they are allowed to do.
- Write Jest and React Testing Library tests for components, and Cypress tests that walk through the repair and approval flow end to end.

Backend:

- Build REST services with Node.js and Express in TypeScript for exception queues, transaction timelines, and reconciliation breaks, split into routes, controllers, services, and repositories.
- Implement the maker-checker rule: a manual repair is not applied until a second user approves it, and both actions are written to an audit table in PostgreSQL.
- Query PostgreSQL with node-postgres and call PL/SQL procedures in Oracle through node-oracledb for reconciliation matching and aging reports, and check execution plans before adding indexes.
- Stop two operators from repairing the same item at once by using a version column in PostgreSQL for optimistic locking.
- Publish and consume payment status and repair events with KafkaJS. Consumers are idempotent, failed messages are retried, and anything that still fails goes to a dead-letter topic for follow-up.
- Wrap calls to downstream systems with timeouts, retries, and a circuit breaker built on the opossum library.
- Secure the APIs with JWT validation middleware and OpenID Connect single sign-on, and map OAuth 2.0 claims to operations, supervisor, and audit roles.
- Validate request bodies with Zod schemas and send every error through one Express error handler so the UI always gets the same error shape.

- Write Jest and Supertest tests for services and endpoints, and document the APIs in OpenAPI so the frontend, QA, and partner teams agree on contracts before coding starts.

Cloud:

- Package each service as a Docker image and release it through a Jenkins pipeline that runs npm install, ESLint, tests, and a SonarQube scan, then deploys to Kubernetes on AWS EKS.
- Troubleshoot failed rollouts, wrong ConfigMap values, and failing health probes on Kubernetes.
- Trace failed transactions across services using Splunk logs and Grafana dashboards, with a correlation ID on every request.
- Keep the SonarQube quality gate green in the Jenkins pipeline by fixing the smells, duplication, and security hotspots it reports.
- Define each service's Kubernetes Deployment, Service, and ConfigMap, run the React build in its own Nginx container, and keep database passwords and API keys in Kubernetes Secrets so nothing sensitive is baked into a Docker image.
- Set Kubernetes readiness and liveness probes and CPU and memory limits on every pod, and release with rolling updates so a deployment does not take the workbench down, rolling back if the new version fails its probes.
- Give each service its own IAM role on AWS EKS so it can reach only the AWS resources it needs.
- Create Grafana alert rules on pod restarts and 5xx error rates so the team hears about a problem before operations users do.

Environment: Node.js 20/22, Express.js, TypeScript, JavaScript (ES6+), REST APIs, OAuth 2.0, OpenID Connect, JWT, Zod, opossum, Apache Kafka, KafkaJS, node-postgres, node-oracledb, PostgreSQL, Oracle, PL/SQL, React 18, Redux Toolkit, React Router, Material UI, Axios, npm, ESLint, Git, Bitbucket, Jenkins, SonarQube, Docker, Kubernetes, AWS (EKS, IAM), Nginx, Splunk, Grafana, Jest, Supertest, React Testing Library, Cypress, Postman, Swagger/OpenAPI, Jira, Agile/Scrum.

Software Developer **May 2024 – Jul 2025**
Amgen

Project description: Clinical Supply Tracking Portal: an internal application that clinical supply and study operations teams used to follow investigational product shipments to trial sites, check inventory at each site, and respond when a shipment went outside its temperature range. Study teams could see where each shipment was, what stock each site held, and which alerts needed action, while supply planners and quality reviewers had their own views. Every change to a shipment record was kept in an audit history, and users signed in through the company's single sign-on.

Frontend:

- Built the React 18 and TypeScript screens for shipment tracking, site inventory, and temperature-excursion alerts, using React.lazy to split routes so each study area loaded only when it was opened.
- Used Redux Toolkit to hold shared state like the selected study, active filters, and user role, and React Router to move between shipment, inventory, and alert pages.
- Built reusable components with Bootstrap and SASS (sortable tables with paging, filter bars, status badges, confirmation modals) and form validation for shipment edits and alert acknowledgments.
- Called the REST APIs through Axios, with an interceptor that attached the access token and dealt with expired sessions and error responses.
- Limited screens by role with React Router route guards driven by OpenID Connect claims, so study teams, supply planners, and quality reviewers saw only what applied to them.
- Wrote Jest and React Testing Library tests for components, plus Cypress tests for the shipment search and alert acknowledgment flow.

Backend:

- Developed Spring Boot 3 REST services on Java 17 for shipments, site inventory, and alerts, with controller, service, and repository layers and DTOs.
- Mapped shipment, inventory, and audit tables with Hibernate and Spring Data JPA, and wrote the audit history (who changed a shipment record, what changed, and when) through a Spring AOP aspect.
- Consumed shipment and temperature-reading events from SQS queues with idempotent handling, so a repeated message could not create a duplicate alert, and published notifications through SNS.
- Secured the APIs with Spring Security as an OAuth 2.0 resource server using OpenID Connect and JWT, and mapped claims to study team, supply planner, and quality reviewer roles.
- Used Bean Validation for input checks and a @ControllerAdvice exception handler to keep error responses consistent.
- Wrote PostgreSQL queries and indexes for shipment search and inventory reports, and fixed slow endpoints by removing N+1 queries with Hibernate fetch joins.
- Wrote JUnit 5 and Mockito tests for services, and Spring Boot test slices for controllers and repositories.
- Documented the endpoints with OpenAPI and tested them in Postman before handing builds to QA.

Cloud:

- Containerized the services with Docker and deployed them to Kubernetes on AWS EKS through Jenkins pipelines that built with Maven, ran tests, and published images.
- Stored shipment documents and exported reports in S3, and used IAM roles to limit what each service could reach.
- Wrote a Lambda function that formatted and sent alert messages, and used CloudWatch logs and alarms to work out what went wrong after a release.
- Served the React build from an Nginx container on EKS and managed environment settings through Kubernetes ConfigMaps.
- Set Kubernetes readiness and liveness probes and resource limits, and released with rolling updates so study teams were not interrupted during a deployment.
- Cleaned up SonarQube findings during pull request reviews in GitHub, and traced defects raised in Jira through Logback logs and PostgreSQL queries.
- Used Amazon RDS automated backups and CloudWatch metrics to keep an eye on database connections and slow queries.
- Created CloudWatch alarms on Lambda errors and SQS queue depth so stuck alert messages were noticed quickly.

Environment: Java 17, Spring Boot 3, Spring MVC, Spring Security, Spring Data JPA, Spring AOP, Hibernate, REST APIs, OAuth 2.0, OpenID Connect, JWT, React 18, Redux Toolkit, React Router, TypeScript, Bootstrap, SASS, Axios, PostgreSQL, AWS (EKS, S3, RDS, SQS, SNS, Lambda, CloudWatch, IAM), Docker, Kubernetes, Nginx, Maven, Git, GitHub, Jenkins, SonarQube, Logback, JUnit 5, Mockito, Jest, React Testing Library, Cypress, Postman, Swagger/OpenAPI, Jira, Agile/Scrum.

Software Engineer **Dec 2021 – Dec 2023**
Credit Suisse

Project description: Private Banking Client Onboarding and KYC Case Portal: an application that relationship managers and compliance analysts used to onboard new private banking clients, collect identity and source-of-funds documents, and move each know-your-customer case through review and approval. Relationship managers completed a guided, multi-step onboarding form, analysts reviewed the documents and recorded their decisions, and approvers gave the final sign-off. Each case moved through defined stages, and its progress was shared with other compliance systems so nothing was missed.

Frontend:

- Built the onboarding flow as a multi-step Angular wizard with reactive forms, questions that appeared depending on client type, and validation messages that said exactly what was missing.
- Developed case dashboards and a review workspace for compliance analysts with Angular Material tables that support sorting, filtering, and paging for long case lists.
- Used RxJS and NgRx to manage case state, selected filters, and the draft of an in-progress onboarding form, so users could leave and come back without losing what they had entered.
- Built reusable Angular Material components such as document upload panels, status badges, and approval dialogs, and shared them across feature areas through common modules.
- Called the Node.js APIs through Angular HttpClient, with interceptors for tokens, error handling, and a global loading indicator, and protected routes with Angular route guards based on user role.
- Helped move the application from Angular 13 up to Angular 16, replacing deprecated APIs and keeping the Jasmine, Karma, and Cypress tests passing.

Backend:

- Built REST APIs with Node.js, Express, and TypeScript for clients, KYC cases, document records, and review decisions, organized into routes, controllers, services, and data-access layers.
- Used Sequelize with PostgreSQL for client and case data, and Mongoose with MongoDB for document metadata and review notes that did not fit a fixed schema.
- Implemented the case workflow as explicit states (draft, submitted, in review, approved, rejected) in the service layer, with checks so a case could only move through allowed transitions.
- Published case events to Kafka with KafkaJS for downstream compliance systems, with retry handling for failed publishes.
- Secured the APIs with JWT validation middleware and OAuth 2.0 scopes, and mapped roles to relationship manager, analyst, and approver permissions.
- Validated request bodies with Zod schemas and returned one consistent error format through central Express error-handling middleware.
- Handled document uploads with Multer, checking file type and size, and kept only the file reference in MongoDB.
- Wrote PostgreSQL queries and indexes for case search and reports, and used async/await with connection pooling in Sequelize so long requests did not block others.
- Documented the APIs with Swagger, tested them in Postman, and wrote Jest and Supertest tests for services and endpoints.

Cloud:

- Wrote Dockerfiles for the Node.js services and for the Angular build, and ran them on Kubernetes.
- Built Jenkins pipelines that installed dependencies with npm, ran ESLint and tests, built the images, and deployed to test and staging environments.
- Served the Angular build through Nginx, with the Nginx container also proxying calls to the Node.js API.
- Checked pod logs and restart events with kubectl when a deployment misbehaved, and adjusted environment variables and resource limits in the Kubernetes manifests.
- Kept database credentials and JWT signing keys in Kubernetes Secrets and other settings in ConfigMaps, so nothing sensitive sat in a Docker image or the repository.
- Ran SonarQube scans in the Jenkins pipeline and cleaned up TypeScript smells and duplicated logic before each release.
- Reviewed pull requests in Bitbucket for the Angular and Node.js code, and worked through QA and production defects raised in Jira using application logs and PostgreSQL and MongoDB records.

Environment: Node.js 16/18, Express.js, TypeScript, JavaScript (ES6+), Angular 13–16, RxJS, NgRx, Angular Material, HTML5, CSS3, SASS, Bootstrap, REST APIs, JWT, OAuth 2.0, Zod, Multer, PostgreSQL, MongoDB,

Sequelize, Mongoose, Apache Kafka, KafkaJS, npm, ESLint, Git, Bitbucket, Jenkins, SonarQube, Docker, Kubernetes, kubectl, Nginx, Jest, Supertest, Jasmine, Karma, Cypress, Postman, Swagger, Jira, Agile/Scrum.

Software Developer **Dec 2018 – Nov 2021**
Voyager IT Solutions, India

Project description: Order and Inventory Management Portal: an application built for a retail and distribution client, used by sales and warehouse teams to enter customer orders, check stock, and track shipments. It calculated order totals, discounts, and taxes, reserved stock so the same items could not be promised twice, and pulled product pricing from the client's ERP system. Warehouse staff used it on tablets as well as desktops to follow orders from entry to dispatch.

- Built the React 16 components for order entry, stock lookup, and shipment tracking, moving to function components and Hooks as the team adopted them, and wrote Jest tests for the key ones.
- Used Redux to keep the current order, customer details, and search filters in one place, and React Router to move between the order, inventory, and shipment pages.
- Added form validation in React for quantities, addresses, and pricing rules, connected the screens to the REST APIs with Axios, and styled the pages with HTML5, CSS3, and Bootstrap so they worked on warehouse tablets as well as desktops.
- Developed Spring MVC and Spring Boot 2.x REST endpoints on Java 8 for customers, orders, inventory, and shipments.
- Wrote the service and DAO layers with Hibernate and JPA against MySQL, with DTOs between the API and the entities, and MySQL queries and indexes for order search and inventory reports.
- Used Java 8 Streams, lambdas, and Optional to simplify filtering and grouping in the order and inventory logic.
- Coded order totals, discounts, and tax calculations with BigDecimal, and stock reservation rules that stopped two orders from claiming the same last items.
- Called the ERP's SOAP pricing service through JAX-WS clients generated from the WSDL, with timeouts and fallback handling.
- Put stock-update messages on RabbitMQ queues so inventory changes did not slow down order entry.
- Wrote JUnit 5 and Mockito tests and checked the REST and SOAP services in Postman and SoapUI before QA handoff.
- Fixed QA defects raised in Jira by reading Log4j2 logs and MySQL records until the root cause showed up.
- Built Maven packages in Jenkins on every merge to main in GitHub and deployed them to Apache Tomcat on test and staging servers, using Docker for test environments and AWS EC2 for staging.

Environment: Java 8, Spring Boot 2.x, Spring MVC, Hibernate, JPA, REST APIs, SOAP, WSDL, JAX-WS, RabbitMQ, React 16, Redux, React Router, JavaScript, HTML5, CSS3, Bootstrap, Axios, MySQL, Maven, Git, GitHub, Jenkins, Docker, AWS EC2, Apache Tomcat, Log4j2, JUnit 5, Mockito, Jest, Postman, SoapUI, Jira, Agile/Scrum.

Education

Master’s in computer science- University at Albany.
Bachelor’s in computer science – JNTUK , Andhra Pradesh, India.